

Combining Numerical Integration and Deep Learning for Rocket Flight Prediction

Paweł Leczkowski, Wiktor Ludwichowski, Emilia Łukasiuk, Alicja Macutkiewicz, Weronika Marszałik
Gdansk University of Technology
Gdansk, Poland

Abstract—This paper presents a hybrid trajectory prediction framework for suborbital rockets operating under telemetry interruptions. The method combines a deterministic fourth-order Runge–Kutta (RK4) baseline with a recurrent neural network based on Gated Recurrent Units (GRUs). Instead of predicting full rocket dynamics directly, the network estimates only the nonlinear residual acceleration not captured by the physical model. The predicted residual is combined with the deterministic baseline and evaluated using an integration-based trajectory consistency loss, which accounts for both acceleration accuracy and accumulated position drift. The model is trained on synthetic telemetry generated from simulated rocket flights. Telemetry interruption is represented during training and evaluation by an encoder-decoder cut-off setup, in which the model observes a historical sensor window and then predicts future acceleration without receiving new telemetry samples. The data generation itself is outside the scope of this paper. The proposed approach is best understood as a physics-informed machine learning framework rather than a classical Physics-Informed Neural Network, since the physical constraint is introduced through trajectory integration rather than through direct differential-equation residuals.

Index Terms—rocket trajectory prediction, numerical integration, Runge–Kutta methods, gated recurrent units, physics-informed machine learning, residual learning, telemetry loss

I. INTRODUCTION

Telemetry loss in suborbital rockets may occur because of communication failure, environmental disturbances, or temporary sensor malfunction. During such events, ground systems lose direct access to the current state of the vehicle, making trajectory reconstruction necessary for mission safety and post-flight analysis.

Classical prediction methods based on numerical integration provide physically interpretable results but cannot fully model nonlinear aerodynamic effects, atmospheric disturbances, or sensor noise. Pure machine-learning approaches can approximate nonlinear dynamics from telemetry data, although they often produce physically inconsistent predictions when operating without constraints.

To address these limitations, this work proposes a hybrid RK4–GRU framework for rocket trajectory estimation. A deterministic RK4 simulator computes the baseline acceleration, while a GRU network predicts only the residual nonlinear component. This reduces the learning complexity and allows the neural network to focus on dynamics that are difficult to model analytically.

The main contributions of this work are: (1) a residual formulation in which a GRU predicts only the acceleration

correction relative to an RK4-based physical baseline, (2) an encoder-decoder forecasting setup designed to emulate temporary telemetry loss, (3) an integration-based physics-informed loss that penalizes accumulated position drift, and (4) a flight-wise synthetic evaluation comparing the proposed hybrid model against non-learning and learning-only baselines.

The present study focuses primarily on synthetic flight data, where full three-dimensional ground truth is available. Validation on real flight telemetry is treated as a separate extension because real sensor data do not provide the same complete 3D ground-truth trajectory.

II. STATE OF THE ART

Trajectory prediction has been an active research topic for decades in aerospace engineering, autonomous systems, and control theory. Existing approaches can be broadly divided into three categories: classical physics-based methods, purely data-driven machine learning models, and hybrid physics-informed frameworks. Each group offers different trade-offs between physical interpretability, computational complexity, and predictive performance.

A. Numerical solutions

Traditionally, rocket trajectory estimation has relied on deterministic mathematical models derived from Newtonian mechanics. In such approaches, the vehicle state is propagated using systems of ordinary differential equations describing thrust, gravity, aerodynamic drag, lift forces, and mass variation during fuel consumption. Since analytical solutions are generally unavailable for realistic flight conditions, numerical integration methods are employed. Among them, Runge–Kutta [9] methods, particularly the fourth-order Runge–Kutta (RK4) scheme, remain one of the most widely used solutions due to their favorable balance between accuracy and computational cost. Classical state estimation methods often combine these models with filtering techniques such as the Kalman Filter (KF), Extended Kalman Filter (EKF), Unscented Kalman Filter (UKF), or Particle Filter (PF), enabling the fusion of noisy sensor measurements with physical models.

Although deterministic approaches provide physically interpretable solutions, their performance strongly depends on the accuracy of the underlying mathematical model. Real rocket flights are affected by aerodynamic uncertainties, atmospheric disturbances, sensor noise, manufacturing tolerances, and control-system corrections that are difficult to model

precisely. Consequently, even small inaccuracies may accumulate over time and lead to significant trajectory deviations, particularly during long prediction horizons or communication interruptions.

B. Data-Driven Sequence Models

The rapid development of machine learning [11] has led to increasing interest in data-driven trajectory prediction methods. Recurrent Neural Networks (RNNs) were among the first deep learning architectures successfully applied to sequential prediction problems. Their ability to maintain an internal hidden state allows them to capture temporal dependencies that cannot be represented by conventional feed-forward neural networks. However, standard RNNs suffer from vanishing and exploding gradient problems, limiting their ability to model long-term temporal relationships.

To address these limitations, more advanced recurrent architectures such as Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs) were introduced. LSTM networks employ dedicated memory cells and gating mechanisms to preserve relevant information over longer time intervals. GRUs achieve a similar objective through a simplified gating structure that requires fewer parameters and lower computational cost. Numerous studies have demonstrated that GRUs often achieve prediction accuracy comparable to LSTMs while reducing training complexity and memory requirements. As a result, GRU architectures have become increasingly popular in embedded systems, real-time forecasting applications, and telemetry analysis, where computational efficiency is an important consideration.

While purely data-driven approaches can successfully learn complex nonlinear relationships directly from telemetry data, they frequently suffer from a lack of physical consistency. Neural networks trained solely to minimize prediction error may generate solutions that violate conservation laws, produce unrealistic accelerations, or accumulate substantial drift during long-term forecasting. This limitation has motivated the development of physics-informed machine learning approaches.

One of the most influential developments in this area is the concept of Physics-Informed Neural Networks (PINNs) [3]. In classical PINNs, governing physical laws are incorporated directly into the optimization objective through differential equation constraints. Instead of learning exclusively from observational data, the network is simultaneously penalized for violating the mathematical equations describing the underlying physical system. This strategy improves generalization and often reduces the amount of labeled training data required.

Since their introduction, PINNs have been successfully applied to numerous scientific and engineering problems, including fluid dynamics, heat transfer, wave propagation, and inverse parameter estimation. However, most PINN formulations focus on solving or identifying systems of differential equations rather than performing sequence forecasting from telemetry streams. Furthermore, PINNs typically require explicit knowledge of the governing equations and often rely on automatic differentiation to compute differential operators.

Such requirements may become challenging when only partial knowledge of the physical system is available or when the objective is future trajectory prediction rather than equation solving.

C. PIML

As a consequence, recent research has increasingly shifted toward broader Physics-Informed Machine Learning (PIML) frameworks. Unlike classical PINNs, PIML encompasses a wider range of techniques for incorporating physical knowledge into machine learning systems. Physical information may be embedded through architectural constraints, hybrid simulation-learning frameworks, regularization terms, conservation laws, or physics-aware loss functions. The key objective is not necessarily to solve differential equations directly, but rather to improve model robustness, interpretability, and physical plausibility.

A particularly promising direction within PIML is residual learning. In residual-learning frameworks, a deterministic physical model provides a baseline prediction, while a neural network learns only the discrepancy between the baseline and observed data. This approach significantly reduces the learning complexity because the network focuses exclusively on modeling effects that are difficult to describe analytically, such as unmodeled aerodynamic phenomena, atmospheric disturbances, or sensor imperfections. Residual-learning approaches have been successfully applied in fluid mechanics, climate modeling, robotics, and aerospace engineering, where they often outperform both purely physics-based and purely data-driven alternatives.

Despite substantial progress in physics-informed learning, relatively little research has addressed the problem of rocket trajectory reconstruction under telemetry interruption using hybrid recurrent architectures. Existing studies typically focus either on deterministic flight dynamics simulations or on direct trajectory prediction using neural networks. To the authors' knowledge, fewer works consider the specific combination of numerical integration, residual acceleration learning, recurrent sequence forecasting, and trajectory-level physical consistency for rocket telemetry interruption scenarios.

III. BACKGROUND

A. Selection of Flight Dynamics Formulation

To model the flight of a suborbital rocket, a foundational physical framework must be selected. For highly dynamic systems, the two primary methodologies are the Newtonian (force-based) and Lagrangian (energy-based) formulations. While both approaches yield analytically identical trajectories, they present distinct trade-offs when integrated into numerical solvers where computational efficiency and accuracy are paramount.

1) *Newtonian Force-Based Formulation:* The classical approach models vehicle dynamics by analyzing the explicit vectors of forces acting upon the rocket. By factoring out the time-varying mass $m(t)$, the deterministic baseline acceleration $\ddot{x}_B$ (1) can be isolated from the complex, unmodeled dissipative

forces $\ddot{x}_S$ (such as aerodynamic drag and lift) (2). Here $T(t)$ represents the engine thrust vector. ω is the angular velocity of the Earth, accounting for non-inertial reference frame forces (Coriolis and centrifugal accelerations). D and L represent the aerodynamic drag and lift forces, respectively.

$$\ddot{\mathbf{x}}_B = \frac{T(t)}{m(t)} \mathbf{d} + \mathbf{g}(\mathbf{R}_0 + \mathbf{x}) - 2\boldsymbol{\omega} \times \dot{\mathbf{x}} - \boldsymbol{\omega} \times (\boldsymbol{\omega} \times (\mathbf{R}_0 + \mathbf{x})) \quad (1)$$

$$\ddot{\mathbf{x}}_S = \frac{1}{m(t)} (\mathbf{D}(\dot{\mathbf{x}}, \mathbf{x}) + \mathbf{L}(\dot{\mathbf{x}}, \mathbf{x})) \quad (2)$$

The primary numerical challenge of the Newtonian framework is its heavy reliance on vector operations. Because variables such as position x , thrust T , and planetary rotation ω are vector-valued, the mathematical framework must rigidly maintain spatial orientations. However, the principal advantage of this formulation is its practical utility for trajectory reconstruction. The immediate output of the equations is spatial acceleration ($\ddot{x}$). This representation is highly intuitive, maps directly to physical telemetry sensors (like onboard IMUs), and can be directly propagated into velocity and position via standard numerical integration schemes without requiring complex coordinate transformations.

2) *Lagrangian Energy-Based Formulation*: Alternatively, the system can be defined via the Lagrangian formalism, which shifts the analytical focus from coordinate vectors to scalar energy states. The system is governed by the classical Lagrange equations of the second kind, incorporating generalized coordinate vectors r alongside generalized thrust forces Q_T and dissipative forces Q_S , where $\mathcal{L}$ is the Lagrangian function (kinetic minus potential energy) and μ is the standard gravitational parameter of the Earth. (3)

$$\begin{cases} Q_B = \frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{r}} \right) - \frac{\partial \mathcal{L}}{\partial r} - Q_T \\ \mathcal{L}(r, \dot{r}, t) = \frac{1}{2} m(t) \|\dot{r} + \omega \times r\|^2 + \frac{\mu m(t)}{\|r\|} \\ Q_T = T(t) \end{cases} \quad (3)$$

$$Q_B - Q_S = 0 \quad (4)$$

While the Lagrangian approach elegantly bypasses vector-force tracking by utilizing scalar energy fields, it introduces a severe numerical bottleneck for real-time trajectory tracking: numerical differentiation. Solving these equations requires computing complex partial derivatives at every time step, which drastically increases computational complexity. Furthermore, the resulting generalized outputs are non-intuitive for flight paths, requiring a secondary mathematical transformation back into spatial coordinates to be usable for trajectory analysis.

Formulation Choice: Consequently, this work adopts the Newtonian formulation. It avoids the computational overhead of numerical differentiation inherent to the Lagrangian approach and outputs explicit accelerations that align seamlessly with both numerical integration algorithms (such as RK4) and recurrent neural network structures.

B. Numerical Integration of Flight Dynamics

As established in the chosen Newtonian formulation, the rocket's flight is governed by a system of differential equations describing the vehicle's acceleration. However, due to the inherent nonlinearity of these equations and time-varying parameters such as the mass $m(t)$, obtaining an analytical solution for the complete model is practically impossible. Therefore, it is necessary to apply numerical integration methods for ordinary differential equations, which make it possible to approximately determine the system trajectory at successive time steps.

Among the available numerical solvers, methods from the Runge-Kutta family [7] are heavily utilized. In these methods, the approximation of the new system state takes into account function values at several intermediate points within a single time step. For this trajectory prediction system, the fourth-order Runge-Kutta method (RK4) is selected. RK4 represents a highly favorable compromise between computational accuracy and computational complexity, making it widely applied in ballistic simulations where high precision is required while maintaining moderate computational cost.

In the practical implementation of the proposed framework, the RK4 numerical integration scheme serves exclusively as the deterministic component of the model. It generates the baseline physical trajectory prediction ($\ddot{x}_B$) based on the defined Newtonian equations, laying the foundation for the machine learning model to estimate the remaining nonlinear residual forces.

C. Sequence-to-Sequence Modeling with GRUs

Sequential telemetry data require a model capable of using information from previous time steps when predicting future vehicle motion. Recurrent neural networks address this problem by maintaining a hidden state, which acts as a compact representation of the temporal context observed so far. This is particularly useful for multivariate rocket telemetry, where acceleration, angular velocity, pressure, and temperature signals evolve jointly over time. In this framework, the recurrent component utilizes Gated Recurrent Units (GRUs) [1]. GRU networks are highly effective for time-series forecasting because they preserve relevant temporal information over long sequences while remaining computationally lighter than other recurrent architectures, such as LSTMs. Rather than detailing the internal gating mechanisms, the critical operational aspect for this application is that the GRU hidden state provides a learned summary of the recent flight history. To utilize this temporal context, the model follows an encoder-decoder sequence-to-sequence structure. This architecture maps naturally to the telemetry-loss scenario considered in this work:

- **The Encoder Stage (Spin-Up)**: The GRU processes a historical telemetry window and updates its hidden state using the available sensor measurements. This hidden state dynamically encodes the current flight regime and the recent evolution of the vehicle dynamics before any communication interruption occurs.

- **The Decoder Stage (Cut-Off):** When a telemetry loss event occurs, no new samples are provided. The model uses the hidden state obtained from the encoder, together with the last available telemetry sample and a cut-off mode flag, to predict the future residual acceleration sequence. Therefore, the GRU does not produce merely a single-step forecast; it generates a complete prediction window of length H , matching the future horizon required during a signal dropout.

By structuring the network in this sequence-to-sequence manner, the data-driven model seamlessly interfaces with the deterministic RK4 component, which simultaneously provides the physics-based baseline trajectory over the same prediction window.

IV. METHODOLOGY

A. Problem Formulation

At each time step t_i , the telemetry data received from the onboard sensors of the suborbital platform are represented by an eight-dimensional vector u_i containing measured acceleration in the X, Y, and Z axes, angular velocity in the X, Y, and Z axes, barometric pressure, and temperature.

The prediction is not formulated as a direct reconstruction of the full vehicle trajectory and dynamics. Instead, the acceleration is divided into two components: a deterministic part obtained from the physical baseline model, denoted as $\ddot{x}_B$, and a nonlinear residual part, $\ddot{x}_R$, estimated by the neural network. Thus, the final acceleration $\ddot{x}_I$ used by the integrated prediction system can be expressed as:

$$\ddot{x}_I = \ddot{x}_B + \ddot{x}_R. \quad (5)$$

Following this formulation, the proposed system is divided into functional segments:

- **Segment D (Data preprocessing)** prepares telemetry sequences used as input to the predictive system.
- **Segment B (Baseline Model)** computes the deterministic RK4 acceleration baseline over the future prediction window.
- **Segment R (Residual Model)** estimates the nonlinear residual acceleration using a GRU-based neural network.
- **Segment I (Integration Layer)** combines both acceleration components and integrates the reconstructed acceleration to evaluate trajectory consistency.

This approach reduces the learning complexity by delegating physically explainable dynamics to the deterministic model, while the neural network focuses on predicting the nonlinear residuals. A simplified diagram demonstrating the data flow between the segments is shown in Fig. 1.

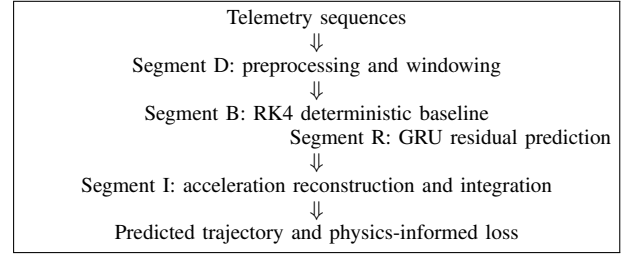


Fig. 1. Simplified information flow in the proposed hybrid prediction system.

B. Data Preprocessing

Segment D takes in telemetry data recorded in a range of different simulated flight scenarios. For each flight, the telemetry vector u_i , the true acceleration $\ddot{x}_{true}(t_i)$, the true position $x_{true}(t_i)$, and the corresponding timestamp t_i are extracted. The sensor signals are downsampled to the effective sampling frequency used by the sequence model. In the current configuration, this reduces the original simulator sampling rate to the temporal resolution used for GRU training and evaluation.

The data are split into training and testing subsets flight-wise. This is important because samples from the same flight are highly correlated. Splitting entire flights rather than random individual windows provides a more realistic evaluation of generalization to unseen trajectories.

Next, Z-score normalization is applied using statistics computed from the training subset:

$$\tilde{f}_i = \frac{f_i - \mu_f}{\sigma_f}, \quad (6)$$

where f_i denotes a given feature value, while μ_f and σ_f denote the mean and standard deviation of that feature computed from the training data. The same statistics are then used to normalize both the training and testing subsets.

Lastly, the data are converted into overlapping training sequences for the GRU model. This is achieved by splitting each flight into a historical observation window and a future prediction window. The historical window is used as the input sequence, while the following H samples form the prediction target. The future acceleration sequence is used to define the residual target, while the future position and timestamps are kept for the physics-informed loss. The last position, estimated velocity, and timestamp before the prediction horizon are stored as initial conditions for numerical integration.

C. GRU Residual Model

Segment R estimates the residual acceleration component $\ddot{x}_R$ using a multi-layered Gated Recurrent Unit. The network receives a historical telemetry data sequence and predicts the residual acceleration over the future prediction horizon. In the current configuration, the prediction horizon H is set equal to the historical input length, although the architecture itself allows other forecast horizons.

It is important to note that the model does not operate on values measured at a single time step independently. Instead, it uses a sequence of past telemetry samples to build temporal

context. Based on information from this sequence, the system predicts the vehicle dynamics over a given future horizon. Thus, at each time step t , the network processes input features x_t and its internal hidden state h_{t-1} .

D. Deterministic Physical Model

Segment B computes the deterministic physical baseline and propagates the vehicle state using a fourth-order Runge–Kutta integration scheme. This baseline is intentionally simplified: it includes engine thrust, changing vehicle mass, launch direction, and gravitational acceleration, but does not explicitly model aerodynamic drag, lift, wind, or other disturbances. These unmodeled effects are treated as part of the residual acceleration learned by the GRU. In the implemented formulation, the deterministic acceleration consists of engine thrust and gravitational acceleration.

The baseline acceleration can be written as

$$\ddot{x}_B(t) = a_T(t) + a_g(t), \quad (7)$$

where

$$a_T(t) = \frac{T(t)}{m(t)}d, \quad a_g(t) = -\frac{GM_E}{\|r(t)\|^3}r(t). \quad (8)$$

In this formulation, $T(t)$ is the engine thrust at time t , $m(t)$ is the current vehicle mass, and d is the launch direction vector. The constants G and M_E denote the gravitational constant and the mass of Earth, respectively. The vector $r(t)$ represents the Earth-centered position of the vehicle, while $\|r(t)\|$ is its distance from the center of Earth. The minus sign in the gravitational term indicates that gravity acts toward the center of Earth.

E. Physics-Informed Integration Layer

Segment I combines the deterministic baseline acceleration with the residual acceleration predicted by the neural network. The reconstructed acceleration is calculated using (5). This acceleration estimate is then integrated over time to obtain the predicted position trajectory.

F. Loss Function and Optimization Objective

The training objective consists of two terms: a residual acceleration error and a physics consistency constraint. The first term directly evaluates the residual acceleration predicted by the GRU network. The second term evaluates whether the total acceleration reconstructed from the baseline and residual components leads to a physically consistent position trajectory after numerical integration.

The total loss is defined as

$$L_{Total} = L_{MSE} + \lambda_{traj}L_{traj}, \quad (9)$$

where λ_{traj} controls the strength of the trajectory-consistency term.

The residual acceleration target is computed by subtracting the deterministic baseline acceleration from the true simulated acceleration:

$$\ddot{x}_{R_{true}}(t_i) = \ddot{x}_{true}(t_i) - \ddot{x}_B(t_i). \quad (10)$$

The residual acceleration loss is then given by:

$$L_{MSE} = \frac{1}{N} \sum_{i=1}^N \|\ddot{x}_R(t_i) - \ddot{x}_{R_{true}}(t_i)\|^2. \quad (11)$$

To compute the trajectory-consistency term, the predicted residual acceleration is first added to the RK4 baseline:

$$\ddot{x}_I(t_i) = \ddot{x}_B(t_i) + \ddot{x}_R(t_i). \quad (12)$$

This reconstructed acceleration is then integrated using Newtonian kinematics to obtain the predicted position $x_I(t_i)$. The trajectory-consistency loss compares this reconstructed position with the true simulated position:

$$L_{traj} = \frac{1}{N} \sum_{i=1}^N \|x_I(t_i) - x_{true}(t_i)\|^2. \quad (13)$$

Unlike classical Physics-Informed Neural Networks (PINNs), which usually penalize differential-equation residuals directly, this framework uses a task-specific trajectory-consistency loss based on numerical integration. Working directly with acceleration keeps the prediction problem rooted in Newtonian dynamics. However, minimizing error in the acceleration space (L_{MSE}) alone is insufficient, as small residual errors accumulate rapidly during numerical integration, leading to substantial position drift. Therefore, the $\lambda_{traj}L_{traj}$ term acts as a trajectory-level constraint. By integrating the reconstructed acceleration and comparing it against the true position, the network is forced to learn residual corrections that are not merely numerically close to the target, but dynamically consistent over the entire prediction horizon.

G. Training Strategy

Training is performed in a sequence-to-sequence manner using overlapping time windows. Each training sample consists of a historical telemetry window and a future prediction window. The historical part is used to initialize the recurrent hidden state, while the future part defines the target residual acceleration sequence.

To simulate the telemetry-loss scenario during training, the model utilizes an internal mode flag with two possible states: Spin-Up and Cut-Off. During the Spin-Up phase, the flag indicates that measured telemetry samples from the historical window are available. The GRU processes this sequence and continually updates its hidden state to encode the current flight regime.

During the Cut-Off phase, which represents the communication interruption, no new telemetry samples are provided. The last available telemetry sample is kept as the decoder input at each future step, while the hidden state carries the temporal information extracted from the historical sequence. A binary mode flag distinguishes the historical Spin-Up phase from the future Cut-Off phase. Relying solely on this internal representation, the network then sequentially generates the future residual acceleration sequence for the entire prediction horizon.

H. Validation Protocol

Model validation is performed using a flight-wise split, ensuring that windows from the same simulated flight do not appear in both training and test subsets. This prevents overly optimistic results caused by temporal correlation between neighboring windows. Evaluation is then performed on unseen flights using both acceleration-space metrics and trajectory-space metrics after numerical integration. This distinction is important because small acceleration errors may accumulate into substantial position drift over the prediction horizon.

V. IMPLEMENTATION

A. Software Pipeline

The proposed model was implemented in Python using PyTorch. The implementation is organized as a modular training pipeline consisting of data loading, sequence construction, deterministic baseline computation, neural prediction, loss evaluation, and checkpointing. The main training script loads simulated flight data from Parquet files, extracts synchronized telemetry and reference states, constructs overlapping sequence windows, trains the GRU-based predictor, and stores model checkpoints together with diagnostic plots.

The implementation supports three related model variants: a direct GRU model, which predicts full acceleration directly; a residual GRU model, which predicts the correction relative to the RK4 baseline; and the full physics-informed residual model, which additionally includes the trajectory-consistency loss. This shared implementation makes it possible to perform ablation experiments under the same data split, normalization procedure, and training configuration.

B. Data Schema and Window Construction

Each simulated flight is stored as a time-indexed Parquet file. The neural network input consists of eight telemetry features: acceleration in the X, Y, and Z axes, angular velocity in the X, Y, and Z axes, barometric pressure, and temperature. The reference targets consist of simulated acceleration, position, and timestamp values. Position is not provided to the neural network as an input; it is used only by the trajectory-consistency term during training and by trajectory-level evaluation metrics.

The raw simulated data are downsampled before training. In the default configuration, the original 500 Hz simulator output is downsampled by a factor of 25, resulting in an effective sampling rate of 20 Hz. Full flights are split flight-wise into training and held-out subsets using an 80/20 split. This avoids placing highly correlated windows from the same flight in both subsets.

After splitting, normalization statistics are computed only from the training subset. Input telemetry, acceleration targets, residual targets, and position values are normalized separately. Each flight is then converted into overlapping sequence pairs: a historical observation window used by the encoder and a future prediction window used as the forecasting target.

C. Deterministic RK4 Baseline

The deterministic baseline is implemented using a fourth-order Runge-Kutta (RK4) solver. The solver propagates the rocket state from an Earth-centered initial position using engine thrust, changing mass due to fuel consumption, launch direction, and gravitational acceleration. The thrust curve and rocket parameters are loaded from the generated physical configuration files.

For each target timestamp, the implementation interpolates the RK4 trajectory to obtain the baseline acceleration $\ddot{x}_B(t)$. In the residual model variants, the training target is computed as the difference between the simulated reference acceleration and this baseline acceleration. Therefore, the neural network is not trained to reproduce the full acceleration field, but only the part not captured by the simplified deterministic model.

D. GRU Architecture

The recurrent predictor is implemented as a custom multi-layer GRU network. The model receives sequences of eight telemetry features and predicts a three-dimensional acceleration sequence. In the final configuration, the model uses two recurrent layers, a hidden size of 64, dropout between recurrent layers, and a linear output layer mapping the hidden representation to the three acceleration components.

An additional binary mode flag is appended internally to the input at each time step. During the encoder, the flag is set to one, indicating that measured telemetry is available. During the decoder, the flag is set to zero, indicating a simulated telemetry cut-off. In this phase, the last available telemetry sample is reused as decoder input while the recurrent hidden state carries the temporal context extracted from the historical window.

E. Loss and Training Configuration

Training is performed using a composite objective. For the residual variants, the acceleration loss compares the predicted residual acceleration with the reference residual obtained by subtracting the RK4 baseline from the simulated acceleration. For the physics-informed variant, the predicted residual is added back to the RK4 baseline and the reconstructed acceleration is integrated using discrete Newtonian kinematics. The resulting position sequence is compared with the simulated reference position, producing a trajectory-consistency penalty.

The total loss is defined as the sum of the acceleration loss and the weighted trajectory-consistency loss. In the final configuration, the trajectory term is weighted by $\lambda_{traj} = 0.2$. The model is optimized using Adam with an initial learning rate of 0.0005 and weight decay of $1e-5$. A plateau-based scheduler reduces the learning rate when the held-out loss stops improving.

F. Checkpointing and Hardware Support

The training pipeline automatically selects the best available compute device: CUDA when available, Apple MPS on compatible systems, Intel XPU when supported, and CPU otherwise. When multiple CUDA devices are available, the model can be wrapped using PyTorch DataParallel.

Training and held-out losses are logged during optimization. The implementation saves periodic checkpoints and also stores the best checkpoint whenever the held-out loss improves. These checkpoints are later used by the evaluation pipeline to compare the direct GRU, residual GRU, and physics-informed residual GRU under identical synthetic test conditions.

VI. EVALUATION METHODOLOGY

The proposed framework was evaluated using a dedicated synthetic test pipeline operating exclusively on held-out flight trajectories. Since consecutive samples within a single flight are strongly correlated, the evaluation does not use a random window-level split. Instead, the dataset is divided flight-wise, so that all windows generated from a given trajectory belong either to the training subset or to the testing subset. This prevents information leakage between training and testing conditions and provides a more realistic estimate of generalization to unseen flights.

Each test flight is converted into overlapping evaluation windows using the same preprocessing procedure as during training. Every window consists of a historical observation interval and a future prediction interval. The historical interval corresponds to telemetry available before a simulated communication loss, while the future interval represents the cut-off period during which the model must continue prediction without receiving new sensor measurements.

The evaluation protocol follows the same encoder-decoder structure used during training:

- **Spin-Up phase:** the model receives a fixed-length telemetry sequence and updates its recurrent hidden state. This phase represents the nominal condition in which recent sensor data are available.
- **Cut-Off phase:** no new telemetry samples are provided. The model predicts the future acceleration sequence using the hidden state obtained during Spin-Up and, for residual variants, the deterministic RK4 baseline over the same future interval.

This procedure reflects the intended use case of the system, where the model must maintain a trajectory estimate after a temporary telemetry interruption. All learning-based and non-learning methods are evaluated over the same test windows and compared against the same simulated reference trajectories.

A. Compared Methods

The evaluation includes both ablation variants of the proposed model and non-learning baselines:

- **Plain GRU:** a direct sequence-to-sequence GRU model predicting full acceleration without using the RK4 baseline.
- **GRU-RK4:** a residual GRU model predicting only the acceleration correction relative to the RK4 baseline, without the trajectory-consistency loss.
- **GRU-RK4 + physics:** the full proposed model, combining residual acceleration learning with the integration-based trajectory-consistency loss.

- **RK4-only baseline:** deterministic propagation using only the simplified physical model, without a learned residual correction.
- **Last-acceleration baseline:** a persistence model assuming that the last observed acceleration remains constant over the prediction horizon.
- **Polynomial extrapolation:** a non-learning statistical baseline fitting a second-order polynomial to the historical position window and extrapolating it into the future.
- **Oracle acceleration:** a diagnostic lower-bound reference that integrates the true future acceleration. This method is not available in real deployment and is used only to quantify the integration error floor.

B. Evaluation Metrics

Model performance is evaluated both in acceleration space and in reconstructed position space. Acceleration metrics verify whether the model predicts the future acceleration signal accurately. Position metrics are treated as the primary criterion, because the practical objective is to maintain an accurate trajectory estimate during telemetry loss.

For a prediction horizon of length H , the 3D position error at time step i is defined as:

$$e_i = \|\mathbf{x}_{pred}(t_i) - \mathbf{x}_{true}(t_i)\|_2, \quad (14)$$

where $\mathbf{x}_{pred}(t_i)$ is the reconstructed position obtained after integrating the predicted acceleration, and $\mathbf{x}_{true}(t_i)$ is the simulated reference position.

For each prediction window, the trajectory RMSE is computed as:

$$RMSE_{window} = \sqrt{\frac{1}{H} \sum_{i=1}^H e_i^2}. \quad (15)$$

Across all windows, we report the global point-wise RMSE, the mean window RMSE, the 95th and 99th percentiles of window RMSE, and the threshold failure rate. A prediction window is classified as failed when:

$$RMSE_{window} > e_{crit}. \quad (16)$$

In the experiments, e_{crit} is set to 10 m. The failure rate is therefore the percentage of prediction windows whose mean trajectory error over the forecast horizon exceeds 10 m.

C. Diagnostic Outputs

In addition to aggregate metrics, the evaluation pipeline generates diagnostic plots for qualitative inspection. These include method-level baseline comparison plots, axis-wise position and acceleration error plots, per-flight error distributions, representative prediction windows, and worst-case trajectory windows selected by the highest 3D window RMSE.

These diagnostic outputs are used to verify whether errors are distributed uniformly across axes, whether failures concentrate in specific flight regimes, and whether the learned residual correction remains stable after numerical integration. The diagnostic plots do not replace aggregate metrics, but provide additional evidence for interpreting model behavior and identifying failure cases.

TABLE I
SYNTHETIC 100-STEP EVALUATION RESULTS ON 412 HELD-OUT FLIGHTS
AND 1,882,588 PREDICTION WINDOWS.

Method	Point RMSE [m]	Mean Window RMSE [m]	p99 [m]	Failure Rate [%]	Acceleration Vector RMSE [$\frac{m}{s^2}$]
Plain GRU	2.99	1.28	16.51	1.660	1.44
GRU-RK4	2.79	1.18	14.86	1.5208	1.40
GRU-RK4 + physics	1.39	0.72	4.23	0.2783	1.02
Polynomial	37.99	9.03	253.13	11.9941	-
RK4 only	39.88	38.37	59.42	93.3541	9.70
Last acceleration	12.51	2.60	54.43	5.4121	4.39
Oracle	0.66	0.27	3.22	0.0001	-

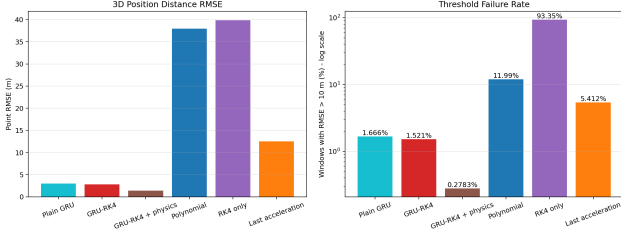


Fig. 2. Comparison of 100-step 3D trajectory error across neural models and non-learning baselines.

VII. SYNTHETIC EVALUATION RESULTS

The final synthetic evaluation was performed using the exhaustive ablation run. The test set contained 412 held-out synthetic flight files and 1,882,588 overlapping prediction windows. Each window used a 100-step forecast horizon, corresponding to approximately 4.3 seconds after downsampling. A prediction window was counted as failed when its 3D window RMSE exceeded 10 m.

A. Short-Horizon Trajectory Accuracy

The main evaluation criterion was 3D position reconstruction over the 100-step forecast horizon. The full GRU-RK4 + physics model achieved the best performance among deployable methods. It reached a 3D point-wise RMSE of 1.390 m, a mean window RMSE of 0.720 m, and a threshold failure rate of 0.2783%.

This result indicates that the model remains stable over the short cut-off interval considered in the main evaluation. The 99th percentile window RMSE was 4.227 m, which means that nearly all evaluated windows remained well below the 10 m failure threshold. In contrast, the deterministic RK4-only baseline produced a mean window RMSE of 38.370 m and a failure rate of 93.354%, showing that the simplified physical model alone is not sufficient for the tested synthetic trajectories.

The last-acceleration baseline performed better than the RK4-only baseline over short horizons, with a mean window RMSE of 2.601 m. However, its 99th percentile error reached 54.436 m, indicating a much larger error tail than the proposed model. The polynomial extrapolation baseline achieved low error for many simple windows, but its 99th percentile error was 253.139 m, showing that purely statistical extrapolation is unstable in difficult trajectory segments.

The oracle acceleration reference achieved a point-wise RMSE of 0.656 m and a mean window RMSE of 0.270 m. Since this reference integrates the true future acceleration, it is not deployable. It is included only to estimate the lower bound imposed by numerical integration, sampling, and evaluation procedure. Its nonzero error reflects the remaining discretization, interpolation, and initial-condition effects in the evaluation pipeline rather than a deployable prediction error.

B. Ablation Study

The ablation study compares three learning-based variants: a plain GRU, a residual GRU-RK4 model, and the full GRU-RK4 + physics model. The plain GRU predicts full acceleration directly and achieved a mean 3D window RMSE of 1.275 m. Adding the residual formulation reduced this value to 1.179 m, showing that residual learning provides a moderate improvement over direct acceleration prediction.

The main improvement appears after adding the integration-based trajectory-consistency loss. The full GRU-RK4 + physics model reduced the mean window RMSE to 0.720 m and reduced the 99th percentile window RMSE from 14.864 m for GRU-RK4 to 4.227 m. The threshold failure rate decreased from 1.521% to 0.2783%.

These results suggest that the trajectory-consistency loss does not merely improve the average prediction error. Its stronger effect is visible in the high-error tail, where integration drift would otherwise accumulate. This is important for telemetry-loss prediction, because isolated acceleration errors can become operationally relevant after being integrated into position.

C. Acceleration-Space Results

Acceleration-space metrics provide additional insight into whether the model learned a meaningful future acceleration signal before integration into position. The full GRU-RK4 + physics model achieved the lowest vector acceleration RMSE among deployable methods, with a vector RMSE of 1.016. The plain GRU and residual GRU-RK4 variants achieved vector RMSE values of 1.436 and 1.401, respectively.

The largest improvement was observed on the Z axis. The full model reached a Z-axis acceleration RMSE of 0.743, compared with 1.222 for GRU-RK4, 1.267 for the plain GRU, 3.741 for the last-acceleration baseline, and 9.505 for RK4-only. Since vertical acceleration has a strong influence on accumulated trajectory error in this dataset, this improvement helps explain the better position-space performance of the full model.

The X and Y acceleration errors were also low for the learning-based models, although the full physics-informed variant did not dominate every individual axis. This suggests that the trajectory-consistency loss primarily improves the overall integrated trajectory and the vertical component, rather than uniformly minimizing every raw acceleration component independently.

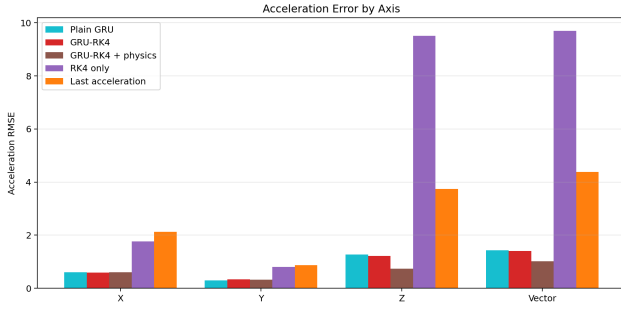


Fig. 3. Axis-wise acceleration RMSE for neural models and acceleration baselines.

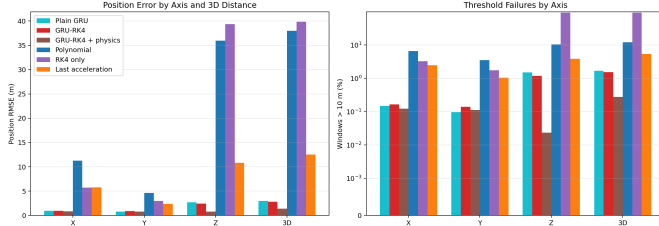


Fig. 4. Axis-wise position reconstruction error for the full GRU-RK4 + physics model.

D. Axis-Wise Position Consistency

The full GRU-RK4 + physics model was also evaluated separately on each spatial axis. The point-wise RMSE values were 0.835 m for X, 0.796 m for Y, and 0.775 m for Z. The corresponding 99th percentile window errors were 2.215 m, 2.010 m, and 3.181 m, respectively.

These results indicate that the improvement is not caused by a single dominant axis. Although the Z axis remains especially important because vertical acceleration strongly affects integrated trajectory drift, the model maintains low reconstruction error across all three spatial dimensions.

E. Diagnostic Analysis

The diagnostic plots provide qualitative support for the aggregate metrics. The method-level comparison shows that the full GRU-RK4 + physics model produces lower short-horizon position errors than the RK4-only, polynomial, and last-acceleration baselines. Axis-wise plots show that the full model remains well balanced across X, Y, and Z position components, with point-wise RMSE values of 0.835 m, 0.796 m, and 0.775 m, respectively.

The per-flight error distribution is useful for checking whether the aggregate results are dominated by a small number of easy trajectories. The low median window error and low failure rate indicate that the model performs consistently across most held-out flights. However, the worst-case windows still show that large errors can occur in difficult flight regimes. These cases are important for future development because they may correspond to high-curvature trajectory segments, underrepresented dynamics, or conditions where the residual

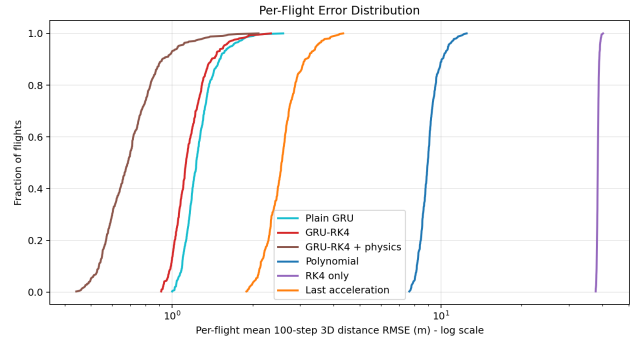


Fig. 5. Per-flight distribution of 3D window RMSE on held-out synthetic trajectories.

model is asked to compensate for too much missing physical structure.

Overall, the synthetic diagnostics support the numerical results: the full model is not only accurate on average, but also substantially reduces the frequency of large short-horizon trajectory failures. At the same time, the diagnostic windows show that rare high-error cases still occur, which defines the main target for future robustness improvements.

VIII. DISCUSSION

A. Interpretation of Synthetic Results

The synthetic evaluation supports three main observations. First, the simplified RK4 baseline alone is insufficient for the tested trajectories. Although it provides a physically interpretable starting point, its thrust-and-gravity model does not capture the full simulated dynamics, leading to large position drift over the prediction horizon.

Second, residual learning is beneficial but not sufficient by itself. The GRU-RK4 residual model outperformed the plain GRU, which suggests that separating deterministic acceleration from learned residual acceleration reduces the complexity of the learning problem. However, the improvement from residualization alone was moderate.

Third, the integration-based physics-informed loss produced the largest gain. The full GRU-RK4 + physics model substantially reduced both average trajectory error and the high-error tail. This indicates that penalizing position drift after integration encourages the network to learn acceleration corrections that are dynamically useful, rather than merely close to the target in acceleration space.

From an operational perspective, the strongest result is the model's short-horizon stability. Over the approximately 4.3 s cut-off window, the full model achieved low mean error and a low failure rate. This matches the intended use case: maintaining a trajectory estimate during a temporary telemetry interruption.

B. Limitations

The main limitation is that the strongest validation is currently synthetic. Although the test flights are held out flight-wise and therefore disconnected from the training windows,

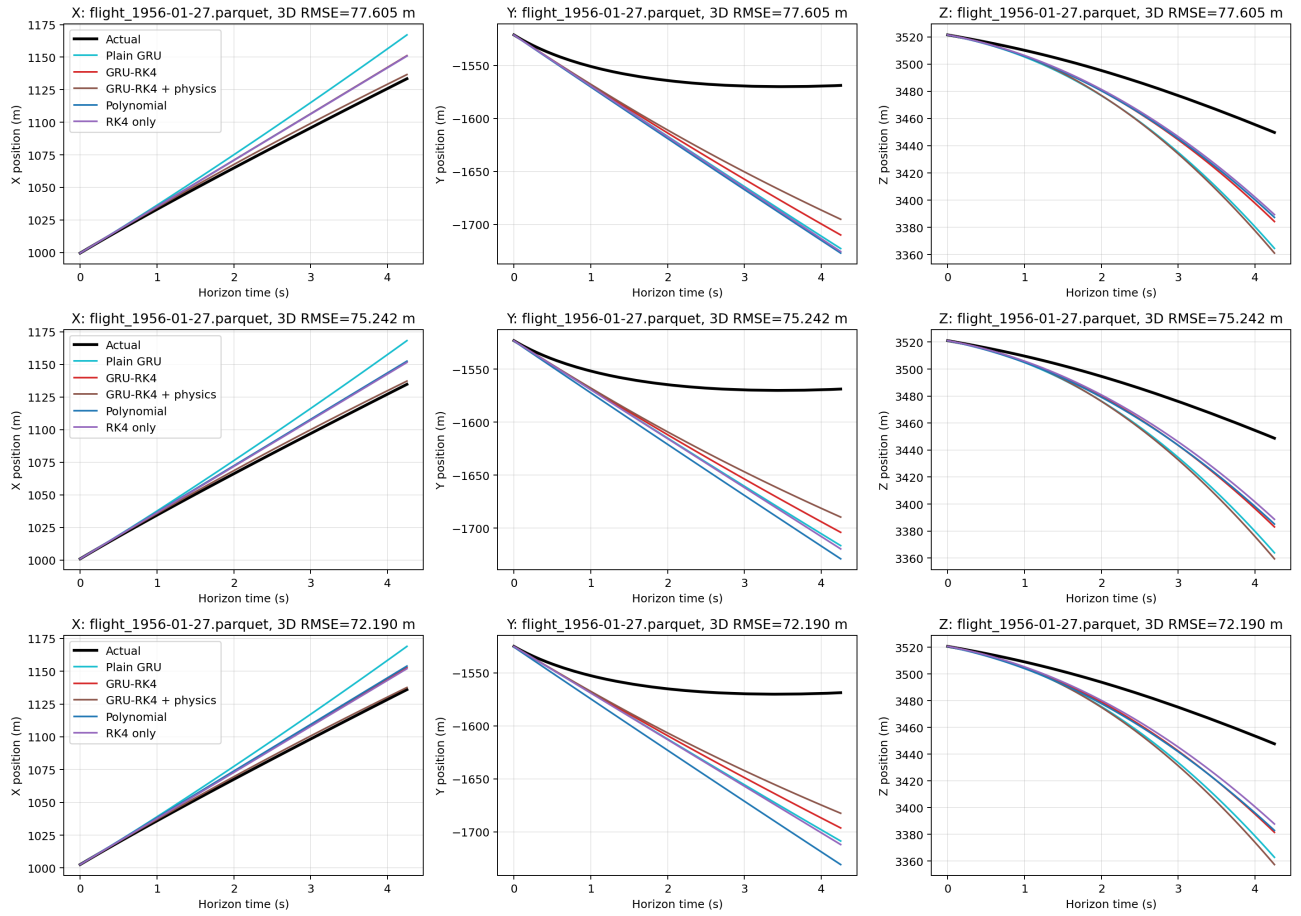


Fig. 6. Worst-case prediction windows selected by highest 3D window RMSE for the full GRU-RK4 + physics model.

they are still generated by the same simulation pipeline as the training data. The results therefore demonstrate generalization to unseen simulated trajectories, not complete validation under real flight conditions.

A second limitation is the forecast horizon. The main evaluation uses a 100-step prediction window, corresponding to approximately 4.3 s. The results should therefore be interpreted as short-horizon telemetry-loss prediction, not as evidence of reliable long-range trajectory or landing prediction. Longer forecast horizons require a separate evaluation protocol and possibly a model trained explicitly for extended prediction.

A third limitation is the simplified deterministic baseline. The RK4 component models thrust, changing mass, launch direction, and gravity, but does not explicitly model all aerodynamic and environmental effects. This is consistent with the residual-learning formulation, but it also means that the neural network may be forced to learn residuals caused by missing physical structure. A more complete physical baseline could reduce the burden on the neural component.

Finally, the current results should not be interpreted as a certified flight safety system. The model provides a promising trajectory-estimation component, but operational deployment would require real-time validation, uncertainty estimation,

robustness testing, and validation on real telemetry.

C. Real-Flight Validation Plan

Real-flight validation should be reported separately from the synthetic evaluation because the available real telemetry does not provide the same full 3D ground truth as the simulator. In synthetic data, acceleration and position references are known exactly for all axes. In real flight data, acceleration can be compared against filtered sensor-derived estimates, and vertical position can be compared against barometric altitude, but full horizontal trajectory accuracy requires additional reference systems or assumptions.

The next validation stage should therefore focus on a Z-axis real-flight evaluation. The model can be tested on real telemetry windows by comparing predicted vertical acceleration against filtered vertical acceleration and integrated vertical position against barometric altitude. This would provide a proof-of-concept real-data validation while clearly distinguishing it from the stronger full 3D synthetic evaluation.

Because the available real flight may differ from the nominal synthetic training distribution, especially in motor configuration, tank state, or launch conditions, domain shift must be handled explicitly. Future experiments should therefore include

retraining or fine-tuning on simulations generated with flight-matched physical parameters, followed by evaluation on the real flight log. The result should be presented as real-flight plausibility validation, not as full 3D ground-truth validation.

IX. CONCLUSION

This paper presented a hybrid rocket trajectory prediction framework combining a simplified RK4-based physical baseline with a GRU residual model and an integration-based trajectory-consistency loss. The synthetic flight-wise evaluation showed that the full GRU–RK4 + physics variant achieved the best short-horizon trajectory reconstruction among deployable methods, reducing both mean window error and high-error tail behavior. The results support the use of residual physics-informed learning for temporary telemetry-loss prediction, while also showing that validation on real flight telemetry and longer forecast horizons remains necessary before operational use.

REFERENCES

- [1] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.
- [2] S. Das and S. Tesfamariam, “State-of-the-art review of design of experiments for physics-informed deep learning,” *arXiv preprint arXiv:2202.06416*, 2022, doi: 10.48550/arXiv.2202.06416.
- [3] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019, doi: 10.1016/j.jcp.2018.10.045.
- [4] S. Cuomo, V. Schiano Di Cola, F. Giampaolo, G. Rozza, M. Raissi, and F. Piccialli, “Scientific machine learning through physics-informed neural networks: Where we are and what’s next,” *Journal of Scientific Computing*, vol. 92, no. 88, 2022, doi: 10.1007/s10915-022-01939-z.
- [5] Y. Wang, J. Sun, J. Bai, C. Anitescu, M. S. Eshaghi, X. Zhuang, T. Rabczuk, and Y. Liu, “Kolmogorov–Arnold-informed neural network: A physics-informed deep learning framework for solving forward and inverse problems based on Kolmogorov–Arnold networks,” *Computer Methods in Applied Mechanics and Engineering*, vol. 433, Art. no. 117518, 2025, doi: 10.1016/j.cma.2024.117518.
- [6] L. M. B. C. Campos and P. J. S. Gil, “On four new methods of analytical calculation of rocket trajectories,” *Aerospace*, vol. 5, no. 3, Art. no. 88, 2018, doi: 10.3390/aerospace5030088.
- [7] Z. A. Anastassi and T. E. Simos, “A trigonometrically fitted Runge–Kutta method for the numerical solution of orbital problems,” *New Astronomy*, vol. 10, no. 4, pp. 301–309, 2005, doi: 10.1016/j.newast.2004.12.003.
- [8] A. Iserles, “Generalized leapfrog methods,” *IMA Journal of Numerical Analysis*, vol. 6, no. 4, pp. 381–392, 1986, doi: 10.1093/imanum/6.4.381.
- [9] J. C. Butcher, “Numerical methods for differential equations and applications,” *The Arabian Journal for Science and Engineering*, vol. 22, no. 2C, pp. 17–29, 1997.
- [10] Y.-L. Cai and Z.-J. Zhou, “A flight trajectory prediction method based on deep learning with attention mechanism,” *Journal of Intelligent Aeronautical Systems and Sustainable Flight Technologies*, vol. 1, pp. 1–11, 2025, doi: 10.65904/3083-3450.2025.01.01.
- [11] M. H. M. Noor and A. O. Ige, “A survey on state-of-the-art deep learning applications and challenges,” *Engineering Applications of Artificial Intelligence*, vol. 159, Art. no. 111225, 2025, doi: 10.1016/j.engappai.2025.111225.